3-Month Software Engineering Internship

Month 1 / Software Engineering Department

Prepared for	Ali Al Hadi Hassan
Role / Position	Full-Stack Developer Intern
Prepared by	HiSoft Agency – Department of Software Engineering
Date	1/10/2026

SMART POS & BUSINESS MANAGEMENT SYSTEM

Project: Smart POS & Business Management System
Duration: 1 Month
Role: Full-Stack Developer Intern
Project Scope: Business Analysis, Database, Backend, Frontend, Security, Testing, Deployment

1. Project Objective

During the first month of the internship, you will work on a **real-world Point of Sale (POS) system**.

The goal is not simply to create screens or APIs.

You are expected to think like a software engineer:

Business Problem → Requirements → Analysis → Architecture → Database Design → API Design → Implementation → Security → Testing → Deployment

You will be responsible for proposing the technical solution.

You will **not** be given the database schema, application architecture, API structure, folder structure, or exact implementation approach.

You must analyze the requirements and decide how the system should be designed.

Every major technical decision must be documented and justified.

2. Business Scenario

HiSoft wants to develop a modern POS system that can be used by small and medium-sized retail businesses.

The business currently has several problems:

- Products are managed manually.
- Employees have different responsibilities but access is not properly controlled.
- Stock quantities are sometimes inaccurate.
- Sales can be entered incorrectly.
- Products may accidentally be sold when the stock is insufficient.

- Different employees may perform operations on the same product at the same time.
- Managers need to know who performed every important operation.
- Customers and their purchase history need to be available.
- Discounts must be controlled.
- Products can be returned.
- Cashiers need to open and close cash sessions.
- Management needs reports about sales and inventory.
- The system must protect sensitive business information.
- The application should be designed so additional modules can be added later.

Your job is to transform this business scenario into a technically solid software solution.

3. Important Rule

Do not start coding immediately.

Before implementing the system, you must first understand the business.

Your first responsibility is to produce:

1. Functional Requirements
2. Non-Functional Requirements
3. User Roles & Permissions
4. User Stories / Use Cases
5. Business Rules
6. Database Design
7. System Architecture
8. API Design
9. Security Considerations
10. Development Plan

The proposed solution must be reviewed before the main implementation begins.

4. Main System Modules

Your architecture should accommodate at least the following areas:

Authentication & Users

The system should support different types of users, for example:

- Administrator
- Manager
- Cashier
- Inventory Employee

You must determine what permissions each role should have.

Products

The system should allow authorized users to manage:

- Products
- Categories
- SKU / barcode
- Purchase price
- Selling price
- Stock quantity
- Minimum stock level
- Product status
- Product information

Think about how the design can support future features such as product variants.

Inventory

The system should track stock changes.

Examples:

- Stock increase
- Stock decrease
- Sale
- Return
- Manual adjustment
- Damaged product
- Initial stock

The system should be able to determine **why the stock changed and who performed the operation**.

Customers

The system should support:

- Customer creation
- Customer information
- Purchase history
- Search
- Customer-specific sales information

You should decide which customer fields are necessary and why.

POS / Sales

The cashier should be able to:

- Search products
- Add products to cart

- Change quantity
- Remove products
- Apply permitted discounts
- Select customer
- Calculate subtotal
- Calculate discount
- Calculate tax if applicable
- Select payment method
- Complete the sale
- Generate a receipt

Payments

At minimum, design the system to support multiple payment methods, such as:

- Cash
- Card
- Other configurable methods

You must decide how payment information should be represented in the database.

Returns / Refunds

The system should support returning previously sold products.

Consider:

- Partial returns
- Full returns
- Returned quantities
- Stock restoration
- Refund amount
- Who approved the return
- Preventing invalid or excessive refunds

Cash Register Sessions

Cashiers should be able to:

- Open a register session
- Enter opening cash
- Perform sales
- Record cash movements
- Close the session

At closing, the system should be able to calculate expected cash.

You must determine how discrepancies should be handled.

Dashboard & Reports

The system should provide basic information such as:

- Total sales
- Number of transactions
- Top-selling products
- Low-stock products
- Sales by payment method
- Sales by cashier
- Sales by date

The architecture should allow future reporting functionality.

Audit Log

Important actions should be traceable.

Examples:

- Login
- Product creation
- Product update
- Price change
- Stock adjustment
- Sale
- Refund
- User/role modification

The audit system should answer:

Who did what, when, and to which record?

5. The Main Engineering Challenges

This project intentionally contains problems where there is no single obvious answer.

You are expected to research, analyze, and propose a solution.

1. **Challenge 1 — Concurrent Stock Updates:**

Imagine:

Product A has **5 units**.

Two cashiers sell the same product at almost exactly the same time.

Cashier 1 sells 4 units.

Cashier 2 sells 3 units.

Your system must prevent the inventory from becoming incorrect.

Your responsibility:

Determine how your backend and database should handle this situation.

Explain:

- What can go wrong?
- Why can it happen?
- How will you prevent it?
- Does your solution require database transactions?
- Do you need locking?
- What happens when stock is insufficient?

You must provide a technical explanation and implement the chosen solution.

2. Challenge 2 — Duplicate Sale

Imagine a cashier presses **"Complete Sale"**.

The network becomes slow.

The cashier presses the button again.

Two requests reach the backend.

You must prevent the system from accidentally creating two identical sales.

You must investigate:

- Idempotency
- Unique transaction identifiers
- Request duplication
- Database constraints
- Frontend protection
- Backend protection

You must decide where the protection should exist and why.

3. Challenge 3 — Price Change

Imagine:

A cashier adds a product to the cart at $10.

The manager changes the product selling price to $12.

The cashier completes the existing cart later.

What price should be used?

You must define the business rule and explain your decision.

The system should preserve the correct historical sale information even after the product price changes in the future.

4. Challenge 4 — Permission and Security

A cashier should not be able to:

- Create an administrator

- Change another user's permissions
- Modify sensitive system settings
- Access management-only reports
- Perform unauthorized refunds
- Change stock without permission

But simply hiding buttons on the frontend is not security.

Your task:

Design a proper authorization system.

You must explain:

What happens if someone directly calls the API?

The backend must enforce authorization independently of the frontend.

5. Challenge 5 — Refund Abuse

Imagine a customer purchased:

5 × Product A

but only wants to return:

2 × Product A

The system must not allow the cashier to refund:

7 × Product A

or refund the same item repeatedly.

Design a solution that prevents invalid refunds.

6. Challenge 6 — Auditability

A manager discovers that the stock of a product changed from:

100 → 64

They need to know what happened.

Your system should make it possible to trace the operations that caused the stock change.

Determine how your inventory history and audit logging should work.

6. Required Architecture Work

You must propose the architecture before implementation.

Your architecture document should explain:

1. **Application Architecture**

Decide how you want to structure:

- Frontend
- Backend
- Database
- Authentication
- Authorization
- Validation
- Error handling
- Logging
- Testing
- Deployment

Explain why you chose your approach.

2. Database Architecture

You must create your own:

- ERD
- Tables
- Relationships
- Primary keys
- Foreign keys
- Indexes
- Constraints
- Transactions considerations

The database should be designed with consistency and future scalability in mind.

3. API Architecture

Define the API structure.

For example, you should determine how the system will handle resources such as:

- Authentication
- Users

- Products
- Categories
- Inventory
- Customers
- Sales
- Payments
- Returns
- Cash sessions
- Reports
- Audit logs

You decide the exact endpoint structure. Document it properly.

7. Technology Expectations

The project should use modern web development practices.

1. **Frontend**

Recommended:

React.js

You should use a clean and maintainable structure.

2. **Backend**

Recommended:

Node.js

You may select an appropriate backend framework and justify your choice.

3. **Database**

Recommended:

PostgreSQL

You must use proper relational database design.

4. **Authentication**

Use a secure authentication mechanism.

You must address:

- Password hashing
- Authentication
- Authorization
- Token/session management
- Expiration
- Logout
- Unauthorized requests

5. **Development Environment**

Use:

- Git
- GitHub
- Environment variables
- Docker where appropriate
- API documentation
- Proper project structure

8. Frontend Requirements

The frontend should include at least:

Authentication

- Login
- Logout
- Protected pages
- Permission-aware UI

Dashboard:

Basic business information.

POS Screen:

The POS screen is one of the most important parts of the project.

It should allow the cashier to:

Search Product → Add to Cart → Change Quantity → Apply Discount → Select Customer → Select Payment → Complete Sale

Product Management:

Authorized users should be able to manage products.

Inventory:

Users with permission should be able to view inventory and stock history.

Customers:

Create and search customers.

Sales:

View previous transactions.

Returns:

Start and process valid returns according to your business rules.

The UI should be responsive and usable rather than simply functional.

9. Backend Requirements

The backend should provide:

- Authentication
- Authorization
- REST API
- Request validation
- Error handling
- Database access
- Transactions
- Business logic
- Inventory management
- Sales processing
- Payment processing
- Returns
- Audit logging
- Logging
- Security controls

Business rules should not be implemented only in React.

Critical rules must be enforced by the backend.

10. Security Requirements

Security is part of the project, not an optional extra.

You must investigate and protect against common web application risks including:

- SQL injection
- Broken authentication
- Broken authorization
- Excessive permissions
- Sensitive data exposure
- Weak password storage
- Brute-force login attempts
- Invalid input
- Malicious payloads
- Insecure API access
- Information leakage
- Improper error messages
- Missing rate limits where appropriate
- Unsafe handling of secrets

You must prepare a basic threat model identifying important attack scenarios and how the system mitigates them.

11. Testing Requirements

The project must contain tests. At minimum, you should investigate and implement:

1. Unit Tests: For important business logic.
2. API / Integration Tests: For critical backend flows.
3. End-to-End Tests: For important user journeys.

At minimum, test scenarios such as:

Login → Product Search → Add to Cart → Complete Sale and Sale → Return → Inventory Restoration

Also test invalid and unauthorized scenarios.

12. Error Handling

The application must not simply crash or expose technical errors to users.

You should design a consistent error-handling strategy.

For example:

- Validation errors
- Authentication errors
- Authorization errors
- Resource-not-found errors
- Business-rule errors
- Database errors
- Unexpected server errors

You must decide how these should be represented by the API and frontend.

13. Deployment

At the end of Month 1, the project should be deployable.

Prepare:

- Production-like environment
- Environment variables
- Database configuration
- Backend deployment
- Frontend deployment
- Basic logging
- README with setup instructions

Someone else should be able to clone the repository and understand how to run the project.

14. Git & Development Process

Use Git professionally.

Every feature should be developed through meaningful commits.

Examples:

feature/authentication

feature/product-management

feature/pos-sales

feature/inventory

Do not submit one massive commit at the end of the month.

Commit messages should clearly describe what was changed.

15. Month 1 Weekly Plan

Week 1 — Analysis & Architecture

Goal:

Understand the problem and design the solution.

Tasks:

- Analyze the business scenario
- Identify actors
- Define user roles
- Define permissions
- Write functional requirements
- Write non-functional requirements
- Define business rules
- Create user stories/use cases
- Identify edge cases
- Identify security risks
- Create ERD
- Design database
- Design application architecture
- Design API
- Compare possible technical approaches
- Create development plan

Deliverables:

1. Requirements Document

2. User Stories / Use Cases

3. Business Rules Document

4. ERD

5. System Architecture Diagram

6. API Specification

7. Security / Threat Model

8. Technical Decisions Document

No major implementation should begin before the architecture is reviewed.

Week 2 — Backend & Database

Goal:

Build the core backend foundation.

Tasks:

- Project initialization
- Database setup
- Migrations
- Authentication
- Password security
- Roles & permissions
- Product module
- Category module
- Inventory foundation
- Customer module
- API validation
- Error handling
- Logging

- Audit logging

Then implement the first version of the sales engine.

The sales operation must correctly handle:

Product → Quantity → Price → Discount → Tax → Payment → Stock

Critical business operations should use appropriate database consistency mechanisms.

Week 3 — Frontend & POS

Goal:

Build the actual user experience.

Tasks:

- Login
- Authentication state
- Protected routes
- Dashboard
- Product management UI
- Inventory UI
- Customer UI
- POS interface
- Shopping cart
- Discounts
- Customer selection
- Payment selection
- Sale completion
- Sales history
- Basic return interface

The POS screen should be designed from the perspective of a real cashier.

The system should not require unnecessary clicks for common operations.

Week 4 — Integration, Security, Testing & Deployment

Goal:

Transform the project from a prototype into a reliable MVP.

Tasks:

- Frontend/backend integration
- Complete critical sales flow
- Complete return flow
- Inventory consistency testing
- Authorization testing
- Security testing
- Unit tests
- Integration tests
- End-to-end tests
- API documentation
- Error-handling review
- Code cleanup
- Performance review
- Dockerization where appropriate
- Deployment
- Final documentation

16. Final Month 1 Deliverables

At the end of the month, you must provide:

Documentation

- Requirements document
- User stories
- Business rules
- ERD
- Architecture diagram

- API documentation
- Security/threat model
- Technical decisions
- Setup instructions

Source Code

- Frontend
- Backend
- Database migrations
- Tests
- Configuration
- Docker files where applicable

Working Application

The application must demonstrate:

Authentication → Authorization → Product → Inventory → Customer → POS → Sale → Payment → Return → Audit

Final Presentation

You must present:

1. The business problem
2. Your requirements
3. Your architecture
4. Database design
5. API design
6. Security approach
7. Important engineering challenges
8. Solutions you selected
9. What you would improve
10. Future scalability plan

17. What We Will Evaluate

Your evaluation will not be based only on whether the application "works."

Engineering & Architecture — 20%

Can you transform a business requirement into a maintainable technical architecture?

Backend — 20%

Code quality, business logic, API design, database interaction and correctness.

Frontend — 15%

Usability, structure, component design and integration.

Database — 10%

Data integrity, relationships, constraints, indexes and transaction handling.

Security — 15%

Authentication, authorization, validation and protection against common attacks.

Testing — 10%

Quality and coverage of important scenarios and edge cases.

Documentation & Communication — 10%

Can another developer understand your system and your decisions?

18. The Most Important Requirement

Do not build the system by following tutorials step-by-step.

You are expected to:

Think → Research → Analyze → Design → Implement → Test → Improve

When you encounter a difficult problem, do not immediately ask: "How do I code this?"

First ask: "What is the actual business problem?"

Then: "What are the possible solutions?"

Then: "Which solution is the safest, simplest, most maintainable, and most scalable?"

And finally: "Why did I choose this solution?"

19. Engineering Mindset Challenge

During this month, the following situations may be introduced without giving you the solution:

- Two cashiers sell the same product simultaneously.
- The same sale request reaches the server twice.
- A cashier attempts to access an administrator API.
- A user modifies the request manually from the browser.
- A product price changes while another cashier has it in the cart.
- A cashier attempts to refund more items than were purchased.
- The server fails during a sale.
- The database transaction partially fails.
- A customer has two similar purchases and wants to return only one.
- Stock is manually adjusted while a sale is being processed.
- A user tries to manipulate the frontend to gain permissions.
- A large number of requests are sent to the login endpoint.

For each problem, you are expected to identify the risk, propose multiple solutions, compare them, select one, and implement it.

20. End-of-Month Definition of Done

Month 1 is considered successfully completed when:

- The architecture is documented.
- The database is properly designed.
- The backend is functional.
- The frontend is functional.
- Authentication and authorization work correctly.
- Core POS operations work.
- Inventory remains consistent.
- Critical edge cases are handled.
- Security has been considered and tested.
- Automated tests exist for important functionality.

- The application is deployable.
- Documentation is complete.
- You can explain every important technical decision you made.

The objective is not to prove that you can write code.

The objective is to prove that you can engineer a software system.

21. HIGHLIGHTED NOTE.

> **Note:**
> **Do not Use Any AI tools to Code**

22. SIGNATURES

For HiSoft Agency™	**Intern**
Signature Name: Rayan Awada Date: 1/10/2026	Signature Name: Ali Al Hadi Hassan Date: 5/10/2026